Manutenção da documentação

> Para quem (pessoa ou agente de IA) for mudar o código depois. Leia antes de editar `src/`, `sim/`, `test/` ou `tools/`.

A documentação mora junto do código, no mesmo repositório, para mudar no mesmo commit que ele. Regra única: **mudou o comportamento, mudou a documentação no mesmo commit.**

Fonte e saída. Os `.md` desta pasta são a **fonte**: é neles que se edita. Os **PDFs** em `pdf/` são a **saída** (para ler, imprimir e entregar) e são gerados a partir dos `.md` por `python3 Documentação/gerar_pdf.py`. Nunca edite um PDF; edite o `.md` e gere de novo, no mesmo commit.

---

1. Onde cada arquivo de código está documentado

Ao alterar um arquivo da coluna da esquerda, revise os documentos da direita.

Arquivo de código	Referência (detalhe por função)	Capítulos (lógica e intenção)
`src/config.h`	referencia/config.md	01 (pinagem), 03, 04, 05, 06 (constantes)
`src/app.h`, `src/main.cpp`	referencia/main.md	02, 04
`src/tilt.h`, `src/tilt.cpp`	referencia/tilt.md	03
`src/posture.h`, `src/posture.cpp`	referencia/posture.md	04
`src/filters.h`	referencia/filters.md	05
`src/mpu.h`, `src/mpu.cpp`	referencia/mpu.md	03
`src/sensors.h`, `src/sensors.cpp`	referencia/sensors.md	03, 05
`src/motor.h`, `src/motor.cpp`	referencia/motor.md	06
`src/settings.h`, `src/settings.cpp`	referencia/settings.md	06
`src/console.h`, `src/console.cpp`	referencia/console.md	07
`src/protocolo.h`, `src/protocolo.cpp`	referencia/protocolo.md	07
`src/web.h`, `src/web.cpp`	referencia/web.md	07
`src/web/index.html`, `manifest.json`, `icon.svg`	—	08
`sim/*`	—	09
`test/test_posture/test_main.cpp`	—	09
`tools/*`	—	09
`platformio.ini`	—	01, 09

Mudou uma **decisão de projeto** (algo listado em 10)? Atualize a decisão ou acrescente uma nova; não apague a antiga, marque como "substituída por D-xx". Mudou algo que o **texto do TCC** afirma (escalas do MPU, filtro, limites)? Atualize 11.

Checklist ao terminar uma mudança

- □ Comentários do arquivo alterado continuam verdadeiros (cabeçalho, funções, blocos).
- □ A página de `referencia/` do módulo bate com as assinaturas atuais (nome, parâmetros, retorno, quem chama).
- □ Capítulos da tabela acima: textos, tabelas de valores e **diagramas** continuam verdadeiros.

- □ Constante nova ou alterada em `config.h`: tabela de referencia/config.md.
- □ Campo novo no JSON ou comando novo: 07 e o comentário do topo de `src/protocolo.h`.
- □ Teste novo: lista de testes em 09.
- □ Rodar a verificação (seção 4).
- □ Mexeu em algum `.md` desta pasta: gerar os PDFs de novo (`python3 Documentação/gerar_pdf.py`) e commitar `pdf/` junto.

---

2. Convenções da documentação

- **Idioma:** português do Brasil, frases curtas, no mesmo tom do `README.md`. Sempre explicar o **porquê**, não só o quê.
- **Arquivos:** nomes em ASCII, minúsculos, com hífen. Capítulos numerados (`NN-assunto.md`); referência por módulo em `referencia/<modulo>.md` (o nome do módulo é o nome do arquivo `.cpp`/`.h`).
- **Documento novo:** acrescentar o caminho na lista `ORDEM` de `ferramentas/gerar_pdf.mjs` (define a ordem no PDF completo e no sumário; o gerador avisa se um `.md` ficou de fora) e uma linha no README.md.
- **Links:** sempre relativos entre os `.md` (`[03](03-algoritmo-inclinacao.md#5-filtro-complementar-sobre-o-vetor-gravidade)`). No PDF completo viram links internos; o gerador para com erro se a âncora não existir. Link para arquivo fora desta pasta (código, `../README.md`) aparece como texto no PDF.
- **Cabeçalho de todo documento:**

  ```
  # Título

  > Código: `src/x.h`, `src/x.cpp` · Relacionados: [03 — Inclinação](03-algoritmo-inclinacao.md)
  ```

  A linha `> Código:` é o elo com o código: um agente que muda `src/x.cpp` encontra o documento com `grep -rl "src/x.cpp" Documentação/`.
- **Citar código por nome, nunca por número de linha** (`TiltFilter::update` em `src/tilt.cpp`). Número de linha muda a cada edição.
- **Unidades sempre explícitas:** °, °/s, g, ms, Hz, %, leitura do ADC (0–4095).
- **Valores numéricos** (limites, períodos, tempos) citam o nome da constante: "20° (`PADRAO_LIMITE_GRAUS`)". Assim dá para achar e atualizar.
- **Termos** (usar sempre os mesmos):

Termo	Significado	No código
desvio	ângulo total entre a gravidade atual e a de referência	`Status::desvio`, `angleBetweenDeg`
frente/trás, lateral	decomposição do desvio, só para exibição	`pitch`, `roll`, `tiltComponentsDeg`
referência (g0)	vetor gravidade gravado na calibração	`Calibracao::g0`
limite	desvio máximo aceito	`Params::limiteGraus`
histerese	margem abaixo do limite para sair do alerta	`Params::histerese`
tempo mínimo	tempo em má postura antes de vibrar	`Params::tempoMs`
curvatura do flex	% de queda da leitura em relação à calibração	`flexPct`
estados	ok, atenção, alerta	`ESTADO_OK`, `ESTADO_ATENCAO`, `ESTADO_ALERTA`
tarefa de controle	tarefa FreeRTOS de sensores, postura e motor	`tarefaControle` em `main.cpp`
loop	`loop()` do Arduino: Wi-Fi e serial	`loop` em `main.cpp`
pedido / evento	mensagens loop → controle / controle → loop	`pedidoCalibracao`, `emitir`

Página de referência de módulo (`referencia/<modulo>.md`)

```
# Módulo `tilt`

> Código: `src/tilt.h`, `src/tilt.cpp` · Relacionados: [03 — Inclinação](../03-algoritmo-inclinacao.md)

## Papel no sistema
O que o módulo faz e por que existe (2–5 frases).

## Dependências
- Usa: ...
- É usado por: ...
- Roda em: tarefa de controle | loop | ambos | PC (testes/simulador)

## Tipos e constantes
Tabela: nome | tipo | descrição | unidade.

## Funções
### `float angleBetweenDeg(Vec3 a, Vec3 b)`
- **O que faz:** ...
- **Parâmetros:** `a`, `b` — ...
- **Retorno:** ...
- **Chamada por:** ...
- **Observações:** detalhes, casos de borda, por que foi feita assim.

## Armadilhas e cuidados
O que pode quebrar se alguém mudar sem cuidado.
```

Diagramas

- Só **Mermaid** dentro do próprio `.md` (bloco ```` ```mermaid ````). É texto: qualquer agente edita junto com o código, e o GitHub e o VS Code desenham.
- Tipos usados: `flowchart`, `sequenceDiagram`, `stateDiagram-v2`. Nada de `%%{init}` nem temas.
- Rótulo com espaço, acento ou símbolo vai **entre aspas**: `A["Filtro complementar<br/>0,98 giro"]`. Quebra de linha no rótulo: `<br/>`. Evite `;` e `#` nos rótulos; em `sequenceDiagram`, evite `;` nas mensagens.
- Uma frase antes de cada diagrama dizendo o que ele mostra.
- No PDF (A4 em pé) o gerador escolhe o tamanho sozinho: um `flowchart LR` muito largo e baixo é desenhado de cima para baixo, e o que ainda ficar largo demais vai sozinho numa página deitada. Se ele avisar "letra pequena no papel", divida ou simplifique o diagrama.
- Figuras para o texto do TCC: ver README.md.

---

3. Convenções dos comentários no código

- Comentários em **português**, estilo `//` (C/C++/JS/CSS usam também `/* */` onde já é o costume do arquivo). Python: docstring no módulo e nas funções, `#` nos blocos. Sem Doxygen.
- **Cabeçalho de arquivo:** o que o arquivo é, o papel no sistema, em que tarefa roda e onde está documentado (`// Documentação: Documentação/referencia/tilt.md`).
- **Toda função, classe, struct e campo de struct** tem comentário: o que faz, unidades dos parâmetros e do retorno, efeitos colaterais e, quando importa, de qual tarefa pode ser chamada.
- **Blocos não óbvios** recebem o **porquê** (fórmula, decisão, armadilha de hardware). Não comentar o óbvio (`i++ // incrementa i`).
- Comentário é parte do código: mudou a lógica, revise o comentário em cima dela.
- Cuidados específicos:

- `src/config.h`: o `VALIDACAO.md` cita linhas deste arquivo (`src/config.h:25`...). Não inserir nem remover linhas antes da linha 66; acrescentar comentário só no fim das linhas existentes ou depois da seção do Wi-Fi.
- `src/web/index.html` vai **inteiro** para a flash e é enviado ao celular a cada carga: comentários curtos. Nunca escrever `</script>` dentro de comentário JS nem `--` dentro de comentário HTML.
- `tools/roteiro*.txt`: comentário é linha começando com `#` **na coluna 0** (o simulador manda para a serial qualquer outra linha, inclusive `# texto`).
- `src/web/manifest.json` é JSON: não aceita comentário.

---

4. Verificação depois de mudar

Com o PlatformIO instalado (PC de desenvolvimento):

```
cd colete-postural
pio test -e native     # testes da lógica pura
tools/sim.sh           # simulador: conferir a saída dos roteiros
pio run                # compila para o ESP32
```

Sem o PlatformIO (só para conferir sintaxe):

```
g++ -std=c++11 -Wall -fsyntax-only -Isrc src/tilt.cpp src/posture.cpp
g++ -std=c++17 -fsyntax-only -DSIMULADOR -Isim -Isrc src/main.cpp src/console.cpp src/sensors.cpp
\
    src/motor.cpp src/settings.cpp src/mpu.cpp sim/sim_main.cpp
python3 -m py_compile tools/*.py
bash -n tools/*.sh
```

Depois de mexer em qualquer `.md` desta pasta (Node.js e Chrome instalados):

```
python3 Documentação/gerar_pdf.py    # gera Documentação/pdf/; termina com erro se um diagrama não
desenhar
                                     # ou um link apontar para âncora que não existe
```

O comando lista os problemas (diagrama com erro de sintaxe, link quebrado) e avisa se um `.md` ficou fora da lista `ORDEM` ou se um diagrama ficou com letra pequena. Confira também, abrindo o PDF, as páginas que você mudou.